

Distributed Systems - Homework 3 Report

Team 32 - Nityanand Gupta (2024101147), A V Aditya (2024111031)

0. Environment

All benchmarks were executed on the RCE SLURM cluster, on compute nodes only (salloc/srun/sbatch), never the login/master node, and spread across multiple distinct physical nodes per the assignment's instructions. Toolchain used on RCE:

- C++: g++ (system default), C++17
- Python: module load python/3.12.5, with grpcio/grpcio-tools 1.84.0 installed to the user site-packages
- Hadoop: /usr/local/apps/hadoop-3.3.0 (currently degraded on RCE - see Section 2.1)

1. Section 1 - Q2: Single Source Shortest Path via Iterative MapReduce

Problem: shortest distance from Node 0 to all nodes in a weighted directed graph.

Design: classic Bellman-Ford relaxation expressed as repeated MapReduce rounds. Each node's state record is node_id <TAB> distance <TAB> adjacency_list.

- Map: re-emits the node's own structure so the graph topology survives to the next round; if its distance is finite, relaxes every outgoing edge and emits a candidate distance to each neighbor.
- Combine: locally collapses multiple candidate distances for the same node to their minimum.
- Reduce: takes the minimum of the previous distance and all incoming candidates, re-attaching the unchanged adjacency list for the next round.

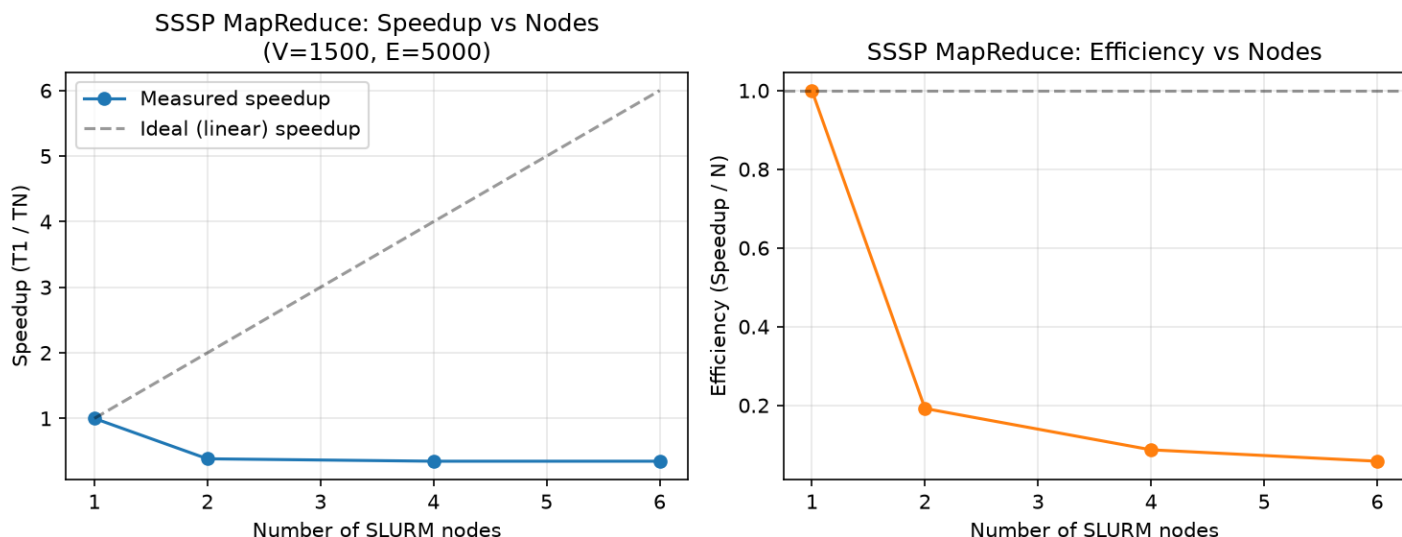
By the Bellman-Ford theorem, after $V-1$ rounds every shortest path (at most $V-1$ edges) has converged; the driver also stops early once a round produces no changes.

Correctness: matches the assignment's sample ($V=4$) exactly, and matches a reference Dijkstra implementation on generated graphs up to $V=1500$.

Verified on RCE (real distributed runs, not local simulation): sample graph ($V=4$) on 2 nodes converged in 3 rounds with an exact output match; a generated graph ($V=1500$, $E=5000$) on 4 nodes converged in 14 rounds (of a possible 1499) with all 1500 distances matching reference Dijkstra.

Scaling study - execution time, speedup, efficiency (same $V=1500/E=5000$ dataset, run at 1/2/4/6 SLURM nodes; time = sum of per-round mapper+shuffle+combiner+shuffle+reducer time across all 14 rounds, excluding SLURM queue overhead):

Nodes	MapReduce time (s)	Speedup	Efficiency
1	2.847	1.000	1.000
2	7.418	0.384	0.192
4	8.205	0.347	0.087
6	8.201	0.347	0.058



Analysis (communication vs. computation): at this problem size, each round's actual per-record work (parse a line, relax a few edges, take a min) is microseconds; round time is instead dominated by srun process-launch and task-coordination overhead. That coordination cost grows with node count while the useful work per task shrinks (a fixed-size problem split into more, smaller pieces), so speedup and efficiency fall well below the ideal linear line and keep falling as N increases. This is not a MapReduce-vs-not-MapReduce problem - it's that per-round srun-based orchestration re-spawns tasks every round, and that fixed cost dwarfs the actual compute at V=1500. It would be expected to amortize better on much larger graphs, or with a persistent-worker runtime (real Hadoop, where the JVMs stay alive across the job) instead of respawning tasks via srun each round.

Deliverables: Section1_MapReduce/SSSP/ (mapper/combiner/reducer, local and SLURM-distributed drivers, dataset generator, correctness checker against Dijkstra, scaling study + plot, README).

2. Section 2: Server Log Analytics (continuing HW2 Q7)

Per the clarified requirement, this section is implemented both ways.

2.1 Q1 - Hadoop MapReduce (C++, Hadoop Streaming)

Design (two stages):

1. Distributed stage - mapper.cpp emits four kinds of single-record partial aggregates (GLOBAL, SERVER_<id>, ENDPOINT_<id>, INTERVAL_<id>). aggregate.cpp serves as both -combiner and -reducer, since every merge (sum/min/max) is associative and commutative and its output is in the same shape as its input. With -numReduceTasks 1, the reducer's output is the complete globally merged aggregate.
2. Local finishing stage - finalize.cpp reuses HW2's own analytics_common.hpp to compute Top-K servers/endpoints, the busiest interval, and format the final output byte-identically to the sequential/MPI reference. By this point the data is only O(S + distinct endpoints + intervals) lines, so a second distributed MR pass would add scheduling overhead for no benefit.

Correctness: verified locally via a Unix-pipeline simulation of the job (mapper | sort | aggregate | sort | aggregate | finalize), matching HW2's expected_output.txt exactly, and matching HW2's own sequential server_log_sequential binary byte-for-byte on a generated 50,000-record dataset.

RCE execution status: blocked by a cluster-wide Hadoop outage. Checked directly on a compute node:

- hdfs dfs -ls / succeeds (reads work), but hdfs dfs -mkdir fails with "Name node is in safe mode" - HDFS cannot currently accept writes.
- yarn node -list hangs indefinitely retrying to connect to the ResourceManager - YARN cannot currently

schedule any job.

This matches the course-wide announcement about a known Hadoop/YARN issue on RCE. `run_hadoop.sh` is complete and ready to run unmodified the moment the environment is restored; until then, correctness is demonstrated via the local pipeline simulation above, which exercises the identical mapper/combiner/reducer/finalize binaries.

MPI (HW2) vs Hadoop MapReduce (this) - design comparison:

Aspect	MPI	Hadoop MapReduce
Data split	Byte-range seek per rank	HDFS input splits
Fixed aggregates	MPI_Reduce	Combiner + reducer
Endpoint/interval keys	MPI_Gatherv, merged	Separate keys, merged by reducer
Top-K / busiest interval	Once, on rank 0	Once, locally, after reduce
Fault tolerance	None	Framework retries tasks
Overhead	Low	Higher (JVM, HDFS, YARN)

2.2 Q2 - Real-Time Streaming Analytics via gRPC

Design: LogAnalyticsService with a client-streaming StreamLogs RPC for ingestion and a GetAnalytics RPC for querying current state. The server shards incoming records round-robin across `num_workers` independent accumulators (default 4), each behind its own lock, so ingestion never contends on one global lock; a query briefly locks each worker in turn to merge a snapshot without blocking ingestion elsewhere. `analytics_core.py` mirrors the C++ aggregation logic field-for-field so all four implementations agree exactly. `streaming_client.py` replays a dataset file at a configurable rate; `dashboard.py` is a polling CLI dashboard.

Correctness (automated test suite, 5/5 passed):

- Full sample-dataset stream produces a final snapshot matching `expected_output.txt` exactly.
- Reset clears all state.
- A 20,000-record stream with concurrent GetAnalytics queries firing throughout ingestion produces no errors, and the final result matches a sequential recomputation.

Verified on RCE with a true 3-node run: server on one node, `streaming_client.py` replaying 2000 records at 500 rec/s from a second node, and a dashboard probe querying from a third node, all communicating over the cluster network rather than localhost:

```
Server node : node01
Client node : node02
Dashboard probe node : node03
[StreamingClient] Ingested 2000 records in 4.003s (499.6 rec/s)
Records ingested so far : 2000   Total requests : 2000
Successful / Failed      : 1154 / 846
[Probe] Queried node01:50252 from a different node than the server -- cross-node gRPC OK.
```

Deliverables: `Section2_RealWorld/Hadoop_MapReduce/` and `Section2_RealWorld/gRPC_Streaming/`, each with its own README.

3. Section 3 - Problem 2: Food Ordering System via gRPC

Design: a central FoodOrderingServicer holds all restaurant/order state in memory, with a lock per Order (fine-grained rather than one global lock). Enforces PLACED -> ACCEPTED -> PREPARING -> READY and PLACED -> CANCELLED, rejecting any other transition. `SubscribeToOrderUpdates` is a server-streaming RPC: the customer immediately receives the current status, then a new update every time the order changes, until a terminal state closes the stream. All required exceptional cases return the appropriate gRPC status code (NOT_FOUND, PERMISSION_DENIED, FAILED_PRECONDITION, INVALID_ARGUMENT).

Correctness (automated test suite, 18/18 passed): restaurant listing, order placement and total calculation,

streamed updates arriving in the correct order, every valid state transition, all 5 required exceptional cases, and 20 concurrent PlaceOrder calls all receiving unique order IDs.

Verified on RCE with a true cross-node run: server on one node, a client probe on a second node connecting over the network rather than localhost:

```
Server node: node06
Client node: node07
[Probe] Order placed: 0101, total=250, status=PLACED
[Probe] UpdateOrderStatus ack: success=True message=Order 0101 -> ACCEPTED.
[Probe] Confirmed status via GetOrderStatus: ACCEPTED
[Probe] Cross-node gRPC communication verified OK.
```

Deliverables: Section3_gRPC/FoodOrdering/ (proto, server, customer CLI, restaurant CLI, automated test, README).

4. Execution Compliance

- Never run on the login/master node. Every benchmark and correctness run above was launched via salloc, srun, or sbatch on allocated compute nodes.
- Multiple physical nodes, not multiple processes on one node. The SSSP distributed runs used 2 and 4 distinct nodes; the Section 2/3 gRPC cross-node demos explicitly placed the server, client, and dashboard/probe on 2-3 different nodes and connected over the cluster network rather than localhost.

5. Summary

#	Section	Implementation	Correctness	RCE Status
1	Sec 1 Q2	SSSP - iterative MapReduce	Matches sample + Dijkstra to V=1500	Verified: 1/2/4/6-node scaling study
2	Sec 2 Q1	Server Log Analytics - Hadoop Streaming	Matches HW2 reference at 10 and 50k records	Blocked by Hadoop/YARN outage; code ready
3	Sec 2 Q2	Server Log Analytics - gRPC streaming	5/5 checks, incl. concurrent ingest+query	Verified: 3-node run
4	Sec 3	Food Ordering - gRPC	18/18 checks	Verified: cross-node run

All source code, READMEs, dataset generators, and correctness/benchmark evidence referenced above are included in this submission.